Sprint 4 — Builder Pattern

Learning Goals

After this sprint, we should be able to:

- Understand why Builder exists.
- Identify the "Telescoping Constructor" problem.
- Compare Constructor vs Setter vs Builder.
- Build immutable objects.
- Use Builder in Java.
- Use Lombok Builder.
- Understand Builder in Spring Boot.
- Know when **NOT** to use Builder.

Class 1

Instead of immediately showing Builder code, we first create the pain.

Lesson 1 — Why Builder Exists

Learning Objective

By the end of this lesson we should be able to answer:

- Why constructors become difficult.
- Why setters are not always good.
- What Builder solves.
- When Builder should be used.

Real Enterprise Example

Imagine we are developing Room Platform.

A room contains:

id
propertyId
roomCode
roomName
roomType
maxAdult
maxChild
price
currency
area
floor
bedType
wifi
parking
kitchen
balcony
airConditioner
television
description
status

Almost 20 fields.

How should we create it?

Option 1

Huge Constructor

```
Room room = new Room(
        id,
        propertyId,
        roomCode,
        roomName,
        roomType,
```

```
        maxAdult,
        maxChild,
        area,
        floor,
        bedType,
        price,
        currency,
        hasWifi,
        hasKitchen,
        hasParking,
        hasBalcony,
        hasAirConditioner,
        hasTelevision,
        description,
        status
);
```

Can you remember which parameter is number 11?

Problems

Very difficult to read.

Easy to swap parameters.

Compile successfully.

Wrong business data.

Example

```
new Room(
    ...
    2,
    5
);
```

Is

2 adults
5 children

or

5 adults
2 children

Nobody knows.

Real Bug

This happens in enterprise projects.

```
new User(
    firstName,
    lastName,
    email,
    phone,
    address,
    role,
    status,
    gender
);
```

Someone accidentally swaps:

status
gender

The compiler says: OK

Business says: Wrong data.

Why Constructors Become Bad

As the object grows:

3 fields -> 6 fields ->10 fields ->18 fields ->25 fields

Constructors become impossible to understand.

Lesson 2

Telescoping Constructor Pattern

Suppose only roomCode is required.

Tomorrow

Business adds

price

Then

currency

Then

wifi

Developers often do this.

public Room(String roomCode)

public Room(

```
    String roomCode,
    BigDecimal price)

public Room(
    String roomCode,
    BigDecimal price,
    Currency currency)

public Room(
    String roomCode,
    BigDecimal price,
    Currency currency,
    boolean wifi)

public Room(...)
```

This is called

Telescoping Constructor

Why It Is Bad

Too many constructors.

Hard to maintain.

Hard to remember.

Impossible after 15 fields.

Question

Have you ever seen a class with 10 constructors?

Usually no.

Lesson 3

JavaBeans Pattern (Setter Pattern)

Most developers improve constructors like this.

```
Room room = new Room();

room.setRoomCode("DLX001");
room.setRoomName("Deluxe");
room.setPrice(BigDecimal.valueOf(50));
room.setWifi(true);
room.setKitchen(false);
```

Looks much cleaner.

Advantages

Readable.

Easy to understand.

Order doesn't matter.

But....

What if someone forgets

```
room.setPrice(...)
```

Now we have

```
Room
```

without price

Invalid object.

Another problem.

Someone can change later.

room.setPrice(null);

Oops.

Builder exists to solve this.

Lesson 4

Classic Builder Pattern

Step 1

Private constructor.

```
public class Room {

    private String roomCode;
    private String roomName;
    private BigDecimal price;

    private Room() {

    }
}
```

Nobody can

```
new Room();
```

Step 2

Builder

```
public static class Builder {

    private final Room room = new Room();

    public Builder roomCode(String roomCode) {
        room.roomCode = roomCode;
        return this;
    }

    public Builder roomName(String roomName) {
        room.roomName = roomName;
        return this;
    }

    public Builder price(BigDecimal price) {
        room.price = price;
        return this;
    }

    public Room build() {
        return room;
    }

}
```

Usage

```
Room room = new Room.Builder()
```

```
        .roomCode("DLX001")
        .roomName("Deluxe")
        .price(BigDecimal.valueOf(50))
        .build();
```

This is much easier to read.

Why It Is Better

Self-documenting.

No parameter order.

Easy to add fields.

Readable.

Lesson 5

Fluent Interface

Builder is actually a Fluent API.

Because every method returns Builder instead of void

Example

```
builder
    .roomCode("DLX001")
    .roomName("Deluxe")
    .price(...)
```

Chain.

Other Fluent APIs we already know

```
Stream.of(...)
        .filter(...)
        .map(...)
        .collect(...)
```

StringBuilder

```
new StringBuilder()
          .append("Hello")
          .append(" Builder")
          .append(" Pattern");
```

Another Fluent API.

End of Class 1

We now understand

Constructors

Setters

Builder

Homework

Current code

```
User user = new User(
          id,
          firstName,
          lastName,
          phone,
          email,
          gender,
```

```
        birthday,
        address,
        active,
        role
);
```

Tasks

Convert to Builder Pattern.

Add

department
photo
language
country

without changing existing client code.

Class 1 Summary

Topic	We Learn
Why Builder Exists	The problem of complex object construction
Telescoping Constructor	Why many constructors become unmaintainable
JavaBeans Pattern	Advantages and weaknesses of setters
Classic Builder	Step-by-step implementation of the GoF Builder
Fluent Interface	Why method chaining makes Builder expressive